

A Major Project Report

A STOCK TRACKER BASIC

Submitted in Partial fulfilment of the requirement for the Award of the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

SAYYAD MUBEENA	(223H1A0577)
N. KUSHIDA	(233H5A0501)
SHAIK MOHASEENA	(223H1A0582)
PONNARUSU HEMACHANDU	(223H1A0569)

Under the esteemed guidance of

Sri. P. Bhaskar, M. Tech, (Ph.D.)

Assistant Professor, Department Of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

K.L.M. COLLEGE OF ENGINEERING FOR WOMEN

(APPROVED By A.I.C.T.E., NEW DELHI AND AFFILIATED TO J.N.T.U. ANANTAPURAMU)

KADAPA-516003

2025-2026

K.L.M COLLEGE OF ENGINEERING FOR WOMEN
(Approved by AICTE and Affiliated to J.N.T.U. Anantapuramu)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"STOCK TRACKER BASIC"** that is being submitted by:

SAYYAD MUBEENA (223H1A0577)

N.KUSHIDA (233H5A0501)

SHAIK MOHASEENA (223H1A0582)

PONNARUSU HEMACHANDU (223H1A0569)

in partial fulfilment of the requirements for the award of the Degree of **Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING** to the **Jawaharlal Nehru Technological University Anantapur** is a record of Bonafide work carried out by under my guidance and supervision. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree.

INTERNAL GUIDE

Sri. P. BHASKAR, M. Tech (Ph.D.)

Assistant Professor

Dept. Of CSE

HEAD OF THE DEPARTMENT

Sri. D. SREENIVASULU, M. Tech (Ph.D.)

Assistant Professor & HOD

Dept. Of CSE

Submitted for university Examination (Viva Voce) held on _____

Internal Examiner

External Examiner

ACKNOWLEDGMENT

We are highly indebted to our superiors for the encouragement and help given for the completion of our project.

We wish to express our gratitude to **Dr. K. CHANDRA OBUL REDDY Managing Director, Kandula Group of Institutions, and Prof. Dr. M.V. RATHNAMMA, Principal of KLM College of Engineering for Women, Kadapa** for their valuable suggestions and advice throughout the course.

At the same time, we express a deep sense of reverence in profound gratitude to our **Head of the Department and Assistant Professor, Sri. D. SREENIVASULU, Department of COMPUTER SCIENCE AND ENGINEERING** and our **Project Guide Sri. P. BHASKAR , Assistant Professor** for their constant motivation, valuable help throughout the project work and for arranging all the facilities, and resources needed for our project. We also extend our thanks to other faculties for their cooperation during our course. Not to forget, our teaching and non-teaching staff and family who had, directly and indirectly, helped and supported us in completing our project.

Project Associates

SAYYAD MUBEENA

N.KUSHIDA

SHAIK MOHASEENA

PONNARUSU HEMACHANDU

VISION AND MISSION OF COLLEGE

VISION

To be an Institution which aims at empowering women through delivering quality Technical Education While Keeping Moral Integrity intact.

MISSION

To provide high standard of Technical Education to women students enabling them to face all challenges of their personal and professional life with courage and confidence.

To make women students as global citizens by inculcating critical thinking, effective communication, robust engineering knowledge and ethical behaviour in them.

To provide an environment that has values and encourages knowledge acquisition, gender equality and academic freedom, making this a preferred Institution for knowledge seekers.

To install self confidence among rural students by supplementing with co-curricular and extra-curricular activities.

VISION AND MISSION OF DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION

To generate competent professionals to become part of the industry and research organizations at the National and International levels.

MISSION

Providing a strong theoretical and practical background across the computer science discipline with an emphasis on software development.

Imparting the skills necessary to continue education to grow professional.
Empowering the women in rural communities with computer education.

Inculcating professional behaviour, strong ethical values, innovative research capabilities and leadership abilities.

TABLE OF CONTENTS

S.No	Title	Page No.
	List of Figures	i
	List of Screens	ii
	Abstract	iii
1	Introduction	1 – 5
1.1	Overview of Project	2
1.2	Problem Statement	3
1.3	Objectives	3
1.4	Scope of the Project	4
1.4.1	Target Audience	5
2	Literature Survey	6 – 8
2.1	Review of Existing Stock Applications	7
2.2	Study of Stock APIs	7
2.3	Analysis of Tracking Algorithms	8
3	System Analysis	9 – 12
3.1	Existing System	10
3.2	Limitations of Existing System	10
3.3	Proposed System	11
3.4	Advantages of the Proposed System	12
3.5	Feasibility Study	12
4	Software and Hardware Requirements	13 – 18
4.1	Software Requirements	14
4.2	Hardware Requirements	15
4.3	Third-Party APIs and Dependencies	16
4.4	SRS (Software Requirements Specification)	17
4.4.1	Functional Requirements	17
4.4.2	Non-Functional Requirements	18
5	System Design	19 – 26
5.1	System Architecture	21
5.2	General Application Flowchart	22
5.3	Use Case Diagram	23
5.4	Class Diagram	24
5.5	Sequence Diagram	25
5.6	Activity Diagram	26
6	Module Implementation	27 – 39
6.1	User Interface Module	28 – 29
6.2	API Integration Module	30
6.3	Data Processing Module	31
6.4	Stock Monitoring Module	32

S.No	Title	Page No.
6.5	Display and Visualization Module	32
6.6	Notification Module	33
6.7	Login & Registration Page	33
6.8	Sample Source Code	34
6.8.1	App.py Code	34 – 36
6.8.2	Dashboard Code	36 – 39
7	Project Structure & Code Implementation	40 – 44
7.1	Flask Project Directory Structure	42
7.2	State Management Implementation	42
7.3	API Integration & JSON Parsing	43
7.4	Main Application Flow	44
8	Testing	45 – 49
8.1	Testing Objectives	46
8.2	Types of Testing Performed	47
8.2.1	Unit Testing	47
8.2.2	Integration Testing	48
8.2.3	System Testing	48
8.2.4	UI Testing	48
8.3	Sample Test Cases	49
9	System Results and Evaluation	50 – 53
9.1	Fulfillment of Objectives	51
9.2	Performance and Efficiency Analysis	52
9.3	Overall Application Outcome	52
10	Future Enhancements	53 – 54
11	Conclusion	55 – 56
12	References	57 – 58

List of figures

S. No	Figure Name	Page No
Fig 1.1	Functional Modules of Stock Tracker System	2
Fig 5.1	System Architecture	21
Fig 5.2	General Application Flowchart	22
Fig 5.3	Use Case Diagram	23
Fig 5.4	Class Diagram	24
Fig 5.5	Sequence Diagram	25

S. No	Figure Name	Page No
Fig 5.6	Activity Diagram of Stock Tracker System	26

List of Screens

Screen No	Screen Title	Page No
Screen 1	Login Page	28
Screen 2	Register Page	29
Screen 3	Dashboard Page	29
Screen 4	Watchlist Page	29
Screen 5	Portfolio Page	30
Screen 6	Trade History Page	30
Screen 7	Help Center Page	30
Screen 8	API Stock Data Fetch Screen	31
Screen 9	Processed Data and Prediction Output	31
Screen 10	Execute Trade and Monitoring Screen	32
Screen 11	Risk Analysis Visualization Screen	32
Screen 12	Notification Alert Screen	33
Screen 13	Profile Page Screen	33
Screen 14	Settings Page Screen	33

ABSTRACT

Stock Tracker Basic is a Python-based application developed to monitor and track real-time stock price changes by using an external stock market API. In today's fast-growing financial world, stock market prices fluctuate continuously, and investors/traders require instant and accurate information to take quick and correct decisions. However, manually checking stock prices through multiple websites or applications is time-consuming, inefficient, and may lead to delayed decision-making.

This project provides an automated solution that simplifies the process of tracking stock market data. The system connects to a stock market API and retrieves live stock details such as current market price, opening price, closing price, day high, day low, trading volume, and overall price fluctuations. The fetched data is processed efficiently and displayed in a clear and structured format, enabling users to easily understand the stock performance.

The primary goal of Stock Tracker Basic is to reduce manual effort, improve data accuracy, and provide real-time stock updates instantly. It helps users to monitor stock movements regularly without repeatedly searching on different platforms. The application is designed with a simple and user-friendly interface, making it suitable for beginners as well as regular market users.

In addition, this system can be expanded further by integrating advanced features such as interactive graphical charts, watchlist functionality, notifications/alerts for price changes, historical stock data analysis, profit-loss calculations, and trend prediction using machine learning techniques. It can also support multiple stock symbols, allowing users to track more than one company at the same time.

Overall, Stock Tracker Basic demonstrates how Python programming and API integration can be effectively used to develop a practical, efficient, and automated stock monitoring system. This project is useful for investors, students, and financial analysts who want an easy way to track and analyze stock market performance in real time.

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

1.1 Overview of the Project

The Stock Tracker Project is designed to monitor and display real-time stock market data in a simple and efficient way. The system focuses on the following major functional areas:

Stock Input Module:

Allows the user to enter a stock symbol (such as TCS, AAPL) to retrieve stock data.

Real-Time Data Fetching:

Fetches live stock market data using APIs like Yahoo Finance (yfinance).

Data Processing Module:

Processes the retrieved data using Python libraries such as pandas and numpy.

Stock Display Module:

Displays important information like stock price, daily change, and market status.

Graph Visualization:

Uses matplotlib to represent stock trends in graphical format for better understanding.

Error Handling System:

Handles invalid inputs or API errors by displaying appropriate messages.

User Interface:

Provides a simple and user-friendly interface for easy interaction.

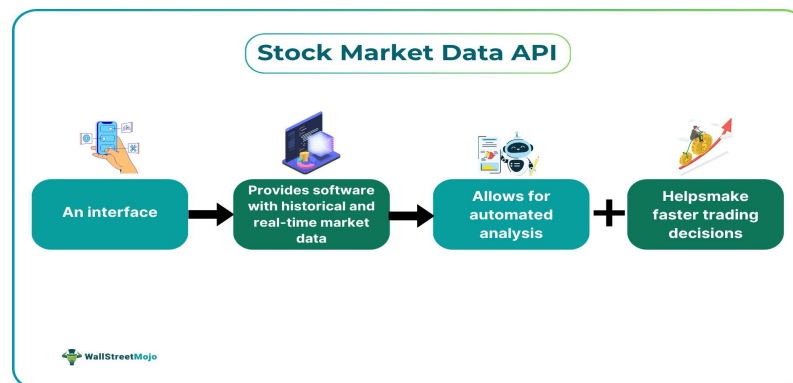


Figure 1.1: Functional Modules of Stock Tracker System

1.2 Problem Statement

In today's rapidly changing financial environment, tracking and predicting stock market trends is a challenging task. Many existing systems either focus only on displaying current stock data or are too complex for beginners to understand. Users often struggle to analyze stock performance and make informed decisions due to the lack of simple tools that provide both real-time data and future predictions.

Additionally, predicting stock prices manually is difficult because it involves analyzing large amounts of historical data and identifying patterns. Without proper tools, users may rely on guesswork or unreliable sources, which can lead to poor investment decisions. Most available applications that offer prediction features are either expensive or require advanced technical knowledge.

Therefore, There is a need for a system that not only tracks real-time stock market data but also predicts future stock trends in a simple and user-friendly manner. The system should use machine learning techniques to analyze historical data and provide approximate future price predictions.

1.3 Objectives

The main objective of the Stock Tracker Project is to develop a system that can track real-time stock data, predict future trends, and assist users in making smart decisions using simple and efficient techniques.

User-Friendly Application: To design a simple and easy-to-use interface that allows users to interact with the system without any complexity.

Real-Time Data Fetching: To retrieve live stock market data using APIs such as Yahoo Finance (yfinance).

Stock Data Display: To present important stock details like current price, opening price, and daily changes in a clear format.

Data Visualization: To display stock trends using graphs for better understanding and analysis.

Prediction of Stock Prices:

To implement machine learning algorithms like Linear Regression to predict future stock prices based on historical data.

Historical Data Analysis:To analyze past stock data and identify patterns useful for prediction.

Auto Buy Feature:To suggest or automatically execute buying of stocks when favorable market conditions are identified.

Auto Sell Feature:To suggest or automatically execute selling of stocks to maximize profit or minimize losses.

Accuracy and Reliability:To ensure that the system provides accurate results and performs reliably.
Error Handling:To handle invalid inputs, API errors, and network issues effectively.
Accessibility for All Users:To make the system suitable for beginners, general users, and small investors.

1.4 Scope of the Project

The scope of the Stock Tracker Project defines the functionalities and limitations of the system while highlighting its potential for future enhancements

Real-Time Stock Tracking: The system allows users to monitor current stock prices by fetching live data using APIs such as Yahoo Finance (yfinance).

Stock Data Display: It provides essential information like current price, daily change, and market status in a simple and understandable format.

Prediction Functionality: The project includes a machine learning model, such as Linear Regression, to predict future stock prices based on historical data.

Auto Buy and Auto Sell Feature: The system can suggest or perform basic automated buy and sell actions based on predefined conditions and predictions.

User-Friendly Interface: The application is designed to be simple and easy to use, making it suitable for beginners and general users.

Educational Purpose: The project is mainly intended for learning and academic use, helping users understand stock tracking, APIs, and machine learning concepts.

Limited Advanced Features: The system does not include complex functionalities such as detailed financial analysis, risk management, or large-scale trading systems.

Dependency on Internet and API: The system requires a stable internet connection and depends on external APIs for real-time data.

Future Enhancements: The project can be extended by adding features such as portfolio management, advanced prediction algorithms, real-time alerts, and mobile or web-based deployment.

1.4.1 Target Audience

The Stock Tracker Project is designed to serve different categories of users who are interested in tracking and analyzing stock market data.

- **Students and Beginners:** The system is useful for students who are learning about stock markets, data analysis, and machine learning concepts. It helps them understand how real-time data and prediction models work.
- **General Users:** Individuals who want to check stock prices and basic market trends can use this application easily without needing technical knowledge.
- **Small Investors:** The project helps small-scale investors monitor stock performance, analyze trends, and make basic investment decisions using real-time data and predictions.
- **Traders (Basic Level):** Beginner traders can use the system to get insights into stock movements and use auto buy/sell suggestions for simple decision-making.
- **Developers and Researchers:** This project is helpful for developers who want to learn about API integration, data processing, and implementing machine learning algorithms like stock prediction.
- **Educational Institutions:** The system can be used in colleges and training institutes as a mini project or reference for learning purposes.

CHAPTER 2

LITERATURE SURVEY

2. LITERATURE SURVEY

The development of the Stock Tracker application is based on the study of existing stock market systems and technologies used for real-time data analysis. Various applications provide stock tracking features using APIs such as Yahoo Finance, which ensure accurate and up-to-date information. Machine learning techniques, especially Linear Regression, have been widely used for predicting future stock prices based on historical data. Several systems also incorporate data visualization tools to represent stock trends effectively. However, many existing solutions lack simplicity, prediction accuracy, and automated decision-making features. Therefore, this project focuses on developing a user-friendly system that integrates real-time tracking, prediction, and auto buy/sell functionalities.

2.1 Review of Existing Stock Applications

The review of existing stock applications shows that many platforms provide real-time stock market data and basic analysis tools. Popular applications such as trading and stock tracking systems use APIs to fetch live stock prices and display them to users. These applications often include features like graphical representation, historical data analysis, and market updates. However, most of them are complex and require prior knowledge to use effectively. Additionally, many existing systems do not provide simple prediction features or automated decision-making options like auto buy and auto sell. Some applications are also paid or have limited access to advanced features. Therefore, there is a need for a simple, user-friendly system that combines real-time tracking, prediction, and basic automation in a single platform.

2.2 Study of APIs

The study of stock APIs is essential for understanding how real-time stock data is fetched and used in applications. Stock APIs such as Yahoo Finance (yfinance), Alpha Vantage, and others provide access to live and historical stock market data. These APIs allow developers to retrieve important information like stock prices, volume, and market trends in a structured format. They are widely used because they offer accurate and up-to-date data, which is crucial for stock tracking and prediction systems. However, some APIs have limitations such as request limits, delayed data, or the need for API keys. Therefore, selecting a reliable and efficient API is important for developing a successful stock tracking application.

2.3 Analysis of Tracking Algorithms

The Stock Tracker Project uses a combination of data processing and machine learning algorithms to provide accurate tracking and prediction of stock prices. Linear Regression is used as the primary algorithm for predicting future stock prices based on historical data. It helps in identifying trends and estimating future values.

The Data Fetching algorithm is responsible for retrieving real-time stock data from APIs such as Yahoo Finance, ensuring that the system has up-to-date information. The Price Change calculation is used to determine the difference between current and previous stock prices, which helps users understand market movement.

Additionally, the Moving Average algorithm is applied to smooth out price fluctuations and identify overall trends in the stock data. This helps in making better predictions and supports decision-making for auto buy and auto sell features. However, these algorithms may have limitations due to market volatility, and prediction accuracy may vary.

CHAPTER 3

SYSTEM ANALYSIS

3.SYSTEM ANALYSIS

3.1 Existing System

The existing stock market systems provide functionalities for tracking and analyzing stock data in real time. These systems use APIs and online platforms to fetch live stock prices and display them to users. Many applications also offer features such as graphical representation, historical data analysis, and market updates. However, most of these systems are complex and require prior knowledge of stock markets, making them difficult for beginners to use. Additionally, they mainly focus on displaying current stock data and do not include simple prediction techniques for future price analysis.

Furthermore, existing systems generally lack automated features such as auto buy and auto sell, which could assist users in making quick decisions. Some applications also require paid subscriptions to access advanced features, limiting their accessibility. In certain cases, the data provided may be delayed or dependent on the reliability of external APIs. Therefore, although existing systems are useful, they have several limitations in terms of simplicity, prediction capability, and automation.

3.2 Limitations of Existing System

The existing stock market systems have several drawbacks that reduce their effectiveness and usability for general users..:

- **Complexity of Use:** Many existing applications are difficult to understand and require prior knowledge of stock markets.
- **Lack of Prediction Capability:** Most systems do not provide simple and accurate stock price prediction features.
- **No Auto Buy/Sell Functionality:** Existing systems generally do not support automated decision-making like auto buying or selling of stocks.
- **High cost:** Some platforms require paid subscriptions to access advanced features and tools.

- **Dependence on Internet and APIs:** The system performance depends heavily on internet connectivity and API availability.
- **Limited User Accessibility:** Many applications are not designed for beginners and are more suitable for experienced traders.
- **Data Delay or Inaccuracy:** Some systems may provide delayed or slightly inaccurate data, affecting decision-making.

3.3 Proposed System

The proposed Stock Tracker system is designed to overcome the limitations of existing systems by providing a simple, efficient, and user-friendly application for tracking and analyzing stock market data. The system enables users to fetch real-time stock prices using reliable APIs such as Yahoo Finance, ensuring accurate and up-to-date information. It presents essential stock details, including current price, price changes, and market trends, in a clear and understandable format.

In addition to real-time tracking, the proposed system incorporates a prediction feature using machine learning algorithms such as Linear Regression. This allows the system to analyze historical stock data and forecast future stock prices, helping users make informed decisions. The system also includes Moving Average calculations to identify trends and reduce fluctuations in stock data.

A key enhancement of the proposed system is the implementation of auto buy and auto sell functionalities. These features assist users by suggesting or executing buy and sell actions based on predefined conditions and predicted trends, thereby improving decision-making efficiency. Furthermore, the system includes proper error handling to manage invalid inputs and API-related issues.

Overall, the proposed system aims to provide a comprehensive solution that combines real-time tracking, prediction, and basic automation in a single platform, making it suitable for beginners, students, and small investors.

3.4 Advantages of the Proposed System

1. **User-Friendly Interface:** The system is simple and easy to use, making it suitable for beginners and students without prior stock market knowledge.
2. **Real-Time Data Access:** It provides accurate and up-to-date stock information using APIs, helping users track market changes instantly.
3. **Prediction Capability:** The system uses Linear Regression to predict future stock prices based on historical data, improving decision-making.
4. **Trend Analysis:** Moving Average calculations are used to analyze stock trends and reduce fluctuations in data.
5. **Auto Buy and Auto Sell Feature:** The system can suggest or perform automatic buying and selling of stocks based on predefined conditions.
6. **Cost-Effective:** It uses open-source tools and free APIs, reducing the overall cost of development and usage.
7. **Error Handling:** The system includes proper error handling to manage invalid inputs and API failures efficiently.

3.5 Feasibility Study

A feasibility study was conducted to ensure the project is practical, sustainable, and capable of being implemented successfully.

Technical Feasibility: The system is technically feasible as it is developed using technologies like Python, APIs, and machine learning algorithms, which are easy to implement and widely available.

Economic Feasibility: The project is cost-effective because it uses free APIs and open-source libraries, requiring minimal investment.

Operational Feasibility: The system is easy to operate and user-friendly, allowing users to quickly understand and use its features without difficulty.

Time Feasibility: The project can be completed within a limited time frame as it uses simple tools and straightforward implementation methods.

Scalability Feasibility: The system can be enhanced in the future by adding advanced features like deep learning models or mobile/web applications.

CHAPTER 4

SOFTWARE AND HARDWARE REQUIREMENTS

4.SOFTWARE AND HARDWARE REQUIREMENTS

The Stock Tracker project application has been developed using a set of modern, open-source, and cross-platform technologies that support efficient development and execution. These technologies provide a reliable environment for building, testing, and deploying the system. The software tools and libraries used in the project ensure smooth data processing, visualization, and implementation of machine learning algorithms.

In addition, the system utilizes third-party APIs for fetching real-time stock market data. The hardware requirements are minimal, making the application accessible on standard computer systems. Overall, these requirements ensure the proper functioning, scalability, and performance of the Stock Tracker system.

4.1 Software Requirements

To successfully develop, compile, and run the Stock Tracker project application, a specific software environment is required. These software components provide the necessary platform for coding, testing, execution, and integration of features such as data fetching, analysis, and prediction. The software requirements are categorized into development software and execution software.

A. Development Environment Requirements The application was built using a cross-platform development environment that supports efficient and flexible system design. It utilizes modern programming tools and open-source libraries to ensure smooth development and integration of features. This environment enables easy testing, debugging, and deployment of the Stock Tracker system across different platforms

- **Programming Language:** The system is developed using Python, which is widely used for data analysis and machine learning applications.
- **Integrated Development Environment (IDE):** Tools like Visual Studio Code and PyCharm are used for writing, testing, and debugging the code
- **Libraries and Packages:** Python libraries such as pandas, numpy, matplotlib, and scikit-learn are used for data processing, visualization, and implementing algorithms.
- **Version Control System:** Git can be used to manage and track changes in the project during development..

B. End-User Execution Requirements (Target Devices) For the final compiled application to run smoothly on a user's mobile phone, the device must meet the following minimum software criteria:

- **Operating System:**system can run on Windows, Linux, or macOS platforms.

- **Web Browser:** Browsers like Google Chrome or Microsoft Edge are used to run and test the application interface.
- **API Integration Software:** The system uses APIs such as Yahoo Finance (yfinance) to fetch real-time stock data.
- **Runtime Environment:** Python interpreter is required to execute the program successfully.

4.2 Hardware Requirements

To ensure smooth development and deployment, the hardware requirements are categorized into two phases: the developer's workstation and the end-user's mobile device.

Development System

Component	Minimum Specification	Recommended Specification
Processor	Intel i3 or equivalent	Intel i5 or Higher
RAM	4 GB	8 GB or Higher
Storage	256 GB	500 GB or higher (SSD preferred)

Target User Device

Component	Requirement
Device Type	Desktop, Laptop, or Smartphone
RAM	Minimum 4 GB (8 GB recommended for better performance)
Sensors	Not mandatory (basic system does not require sensors)

Component	Requirement
Network	Stable internet connection (Wi-Fi or Mobile Data)

4.3 Third-Party APIs and Dependencies

The Stock Tracker project application relies on third-party APIs and external dependencies to fetch real-time stock market data and perform data analysis efficiently. These APIs and libraries play a crucial role in providing accurate information, processing data, and implementing machine learning algorithms. Proper integration of these components ensures smooth functioning and reliable performance of the system.

Libraries, APIs, and Frameworks

S.NO	API/Dependency	Version	Categorry	Description
1	Yahoo Finance (yfinance)	Latest	API	Fetches real-time And historical stock data
2	pandas	Latest	Data Processing Library	Used for data manipulation and analysis
S.NO	API/Dependency	Version	Categorry	Description

3	numpy	Latest	Numerical Library	Used for mathematical computations
4	matplotlib	Latest	Visualization Library	Used for graph plotting and charts
5	scikit-learn	Latest	Machine Learning Library	Used for implementing Linear Regression

4.4 SRS (Software Requirements Specification) Document

The Software Requirements Specification (SRS) document provides a detailed description of the functional and non-functional requirements of the Stock Tracker project application. It defines how the system should behave, the features it must include, and the performance standards it should meet. The SRS helps in understanding the system requirements clearly and acts as a reference throughout the development process.

4.4.1 Functional Requirements The functional requirements describe the core features and operations of the system that allow users to interact with the application.:

- **Stock Input:** The system should allow users to enter a stock symbol to retrieve data.
- **Data Fetching:** The system should fetch real-time stock data using APIs such as Yahoo Finance.
- **Data Display:** The system should display stock details like current price, price changes, and trends..
- **Prediction Feature:** The system should predict future stock prices using Linear Regression.

- **Trend Analysis:** The system should calculate moving averages to analyze stock trends
- **Auto Buy Feature:** The system should suggest or perform automatic buying of stocks based on predefined conditions.
- **Auto Sell Feature:** The system should suggest or perform automatic selling of stocks based on predefined conditions.
- **Error Handling:** The system should handle invalid inputs and API errors effectively.

4.4.2 Non-Functional Requirements The non-functional requirements define the quality attributes and performance criteria of the system.:

- **Performance:** The system should provide fast response time while fetching and displaying data.
- **Accuracy:** The system should ensure accurate data retrieval and reliable prediction results.
- **Usability (UI/UX):** The system should be user-friendly and easy to navigate.
- **Compatibility:** The system should work on multiple operating systems such as Windows, Linux, and macOS.
- **Security:** The system should ensure safe usage of APIs and protect data from unauthorized access.
- **Scalability:** The system should support future enhancements and additional features.
- **Reliability:** The system should function consistently without failures.
- **Availability:** The system should be available whenever required, with minimal downtime.
- **Maintainability:** The system should be easy to maintain and update with new features or bug fixes.
- **Portability:** The application should be easily portable across different platforms and environments.
- **Efficiency:** The system should use minimum system resources such as memory and CPU.
- **Flexibility:** The system should allow modifications and improvements without affecting existing functionality.

CHAPTER 5

SYSTEM DESIGN

5. SYSTEM DESIGN

The System Design defines the structure and workflow of the Stock Tracker application .It consists of modules like user interface, data fetching, processing, and visualization. The system fetches real-time stock data using APIs, processes it using python libraries,and displays it in both text and graphical formats. The design ensures efficiency, accuracy, and ease of use.

5.1 System Architecture

The system architecture shows how different components of the Stock Tracker application interact with each other. It explains the flow of data from the user interface to the backend, APIs, database, and finally to the dashboard.

User (UI):

The user interacts with the system through the dashboard by entering a stock symbol. The interface is designed to be simple and user-friendly, allowing easy access to stock information and analysis.

Backend (Server):

The user request is sent to the backend developed using Flask and Python. The backend handles the request, validates the input, and manages communication between different components of the system.

API (Data Source):

The backend fetches real-time stock data from external APIs such as Yahoo Finance. This includes important details like stock price, high, low, and volume.

Database (SQLite):

The fetched data is stored in an SQLite database. This helps in maintaining historical data and improves performance by reducing repeated API calls.

Processing & Visualization:

The system processes the collected data using Python libraries like pandas and numpy. It also generates graphical representations using matplotlib to help users understand trends easily.

Output (Dashboard):

Finally, the processed data and charts are displayed on the dashboard. The user can view stock details, analyze performance, and make decisions based on the information provided.

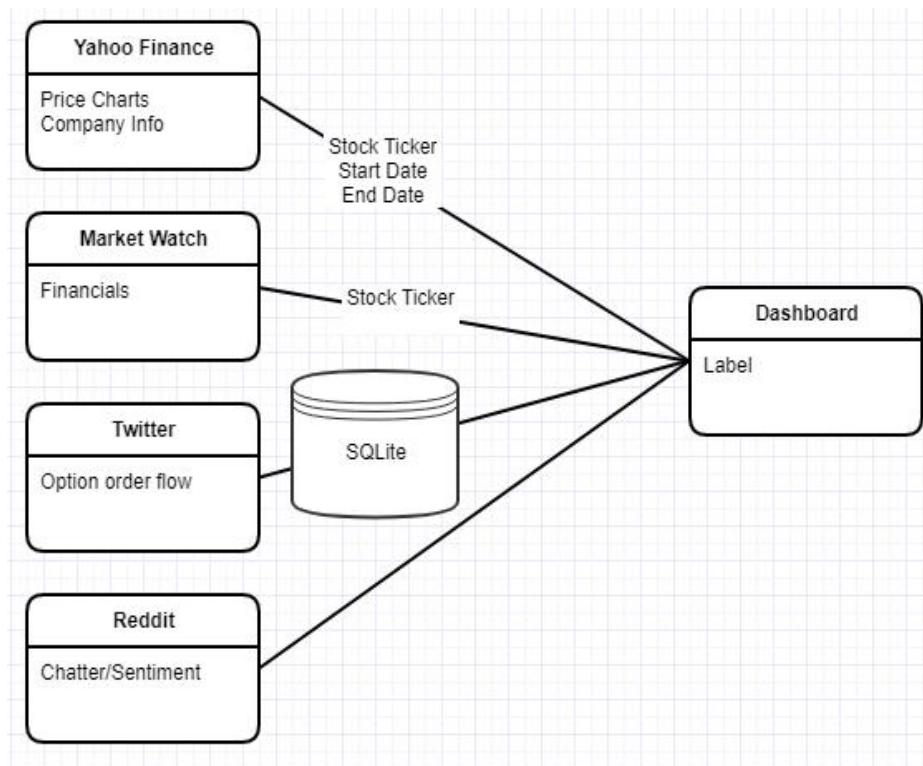


Fig. 5.1 System Architecture

5.2 General Application Flowchart

The General Application Flowchart represents the step-by-step working process of the Stock Tracker system. It shows how the application handles user input and processes data to generate the final output.

Initially, the user enters a stock symbol through the dashboard. The system validates the input and checks whether it is correct or not. If the input is valid, the system fetches real-time stock data from external APIs such as Yahoo Finance.

The fetched data is then processed using Python libraries to extract useful information and generate graphs. Finally, the processed data and visualizations are displayed to the user on the dashboard. If the input is invalid, the system shows an appropriate error message.

Fig. 5.2 General Application Flowchart

5.3 Use Case Diagram

- **User:**
The user is the main actor of the system.
All operations like login, searching stocks, trading are performed by the user.
- **Login & Register:**
The user can create a new account using the register option.
After registration, the user logs into the system securely.
- **Search Stock:**
The user enters a stock symbol (like TCS, AAPL).
The system fetches real-time stock data and displays it.
- **Trade:**
The user can perform buy or sell operations on selected stocks.
The system calculates investment, profit, and loss.

- **Auto Buy & Auto Sell Signal:**
The system automatically generates buy/sell signals.
Based on conditions, trades can be executed automatically.
- **View Dashboard:**
The dashboard shows stock details, charts, and signals.
It helps the user understand market trends easily.
- **View Trade History:**
The user can view previous trades.
This helps in tracking performance and analysis.

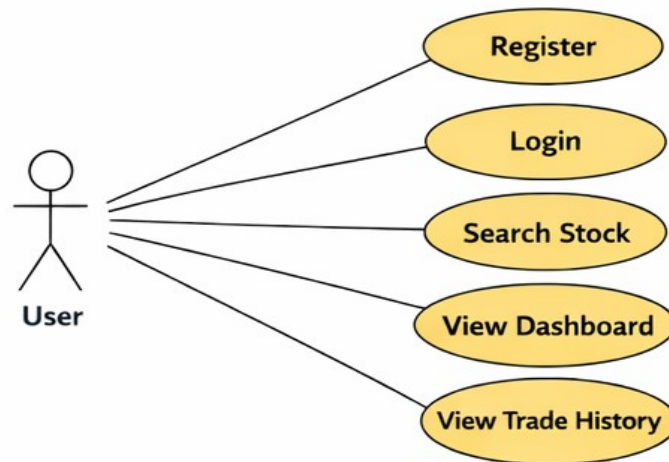


Fig. 5.3 Use Case Diagram

5.4 Class Diagram

The Class Diagram shows the static structure of the Stock Tracker system by detailing its classes, attributes, and operations. The primary classes include:

- **User:** Stores user details like id, name, email, and password.
- **Stock:** Maintains stock information such as symbol, price, and volume.
- **Trade:** Handles buy and sell operations and stores trade details.
- **AutoTrade:** Generates buy/sell signals and performs automatic trading.
- **API Handler:** Manages API requests and fetches real-time stock data.
- **Database:** Stores user data, stock data, and trade history.

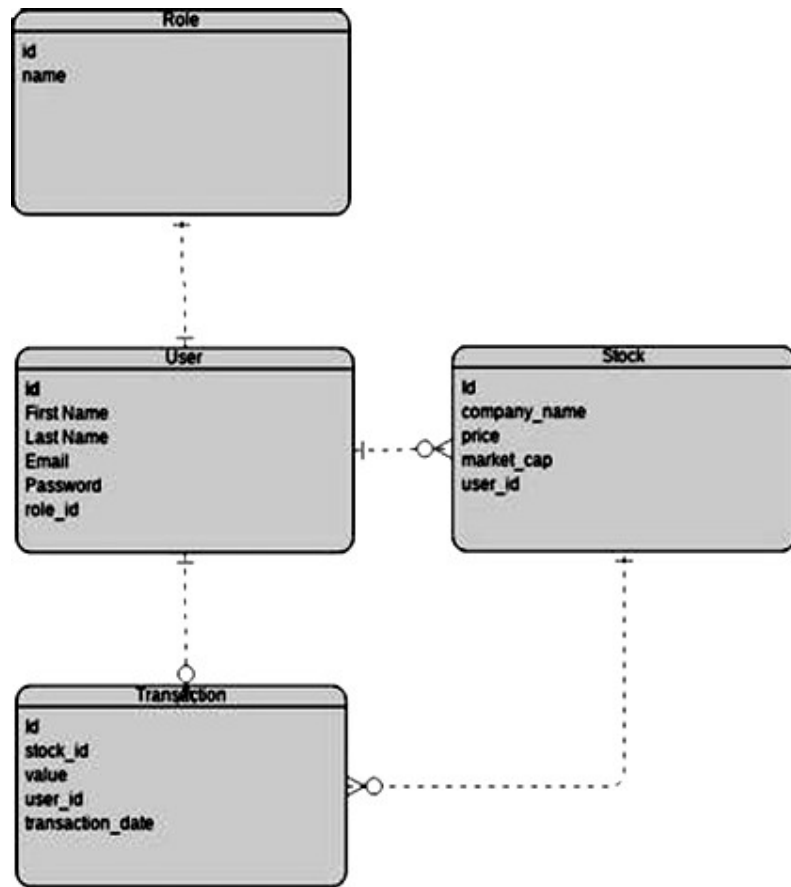


Fig. 5.4 Class Diagram

5.5 Sequence Diagrams

The Sequence Diagram shows the interaction between different components of the Stock Tracker system in a time sequence. The main components include:

- **User:** Initiates the request by entering a stock symbol.
- **User Interface:** Sends the request to the backend.
- **Backend:** Processes the request and calls the API.
- **API Handler:** Fetches stock data from external sources.
- **Database:** Stores and retrieves data.
- **Processing Module:** Analyzes data and generates signals.
- **Dashboard:** Displays the final output to the user.

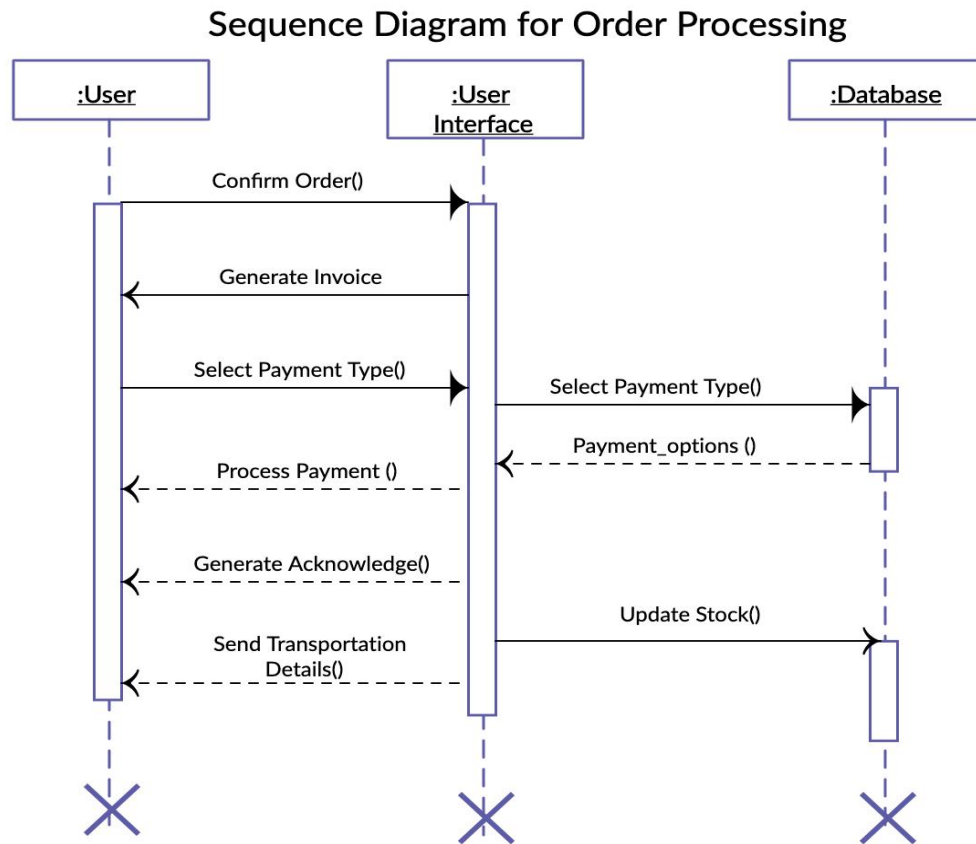


Fig. 5. 5 Sequence Diagram

5.6 Activity Diagram

The Activity Diagram represents the workflow of the Stock Tracker system from start to end. It shows how the user enters a stock symbol, the system validates the input, and fetches data from the API.

The data is processed to generate buy/sell signals. Based on the result, the system performs auto buy or auto sell operations. Finally, the output is displayed on the dashboard, or an error message is shown if the input is invalid.

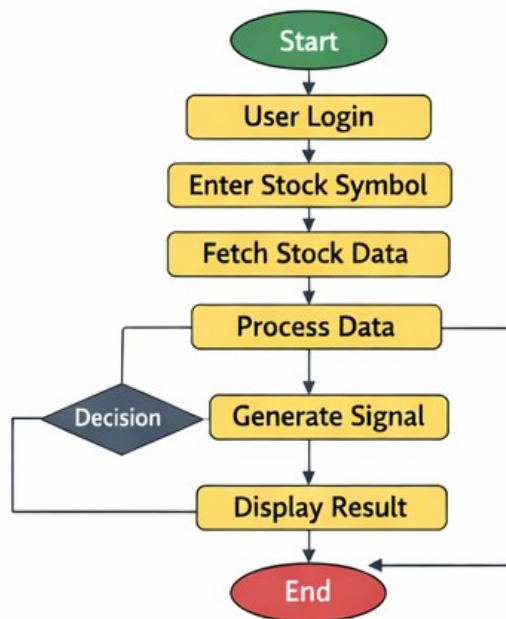


Figure 5.6: Activity Diagram of Stock Tracker System

CHAPTER 6

MODULE IMPLEMENTATION

6. MODULE IMPLEMENTATION

This chapter describes the implementation details of the Stock Tracker Basic system. The project is developed using **Python Flask framework** with HTML, CSS and JavaScript for frontend design. The system mainly focuses on fetching real-time stock market data, processing it, monitoring stock price movements, and displaying results in a user-friendly dashboard. Each module is implemented separately and later integrated to work as a complete system.

6.1 User Interface Module

The User Interface module provides the front-end design of the application. It is developed using HTML, CSS, and JavaScript. This module allows users to login, register, search stocks, view charts, manage watchlist, portfolio, trade history, and access help center.

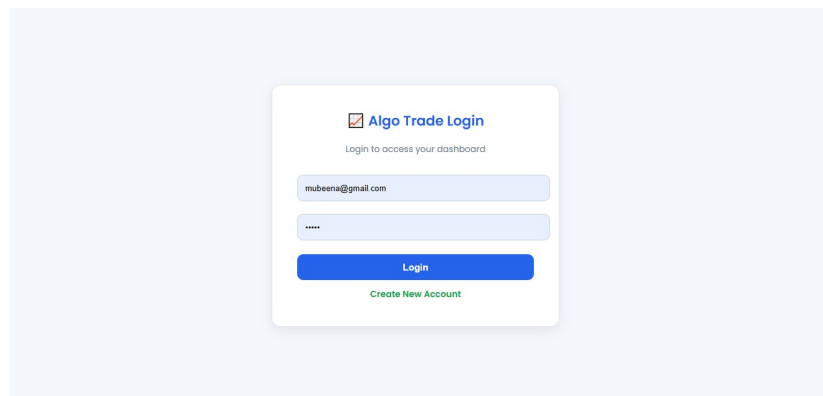


Figure 6.1.1: Login Page

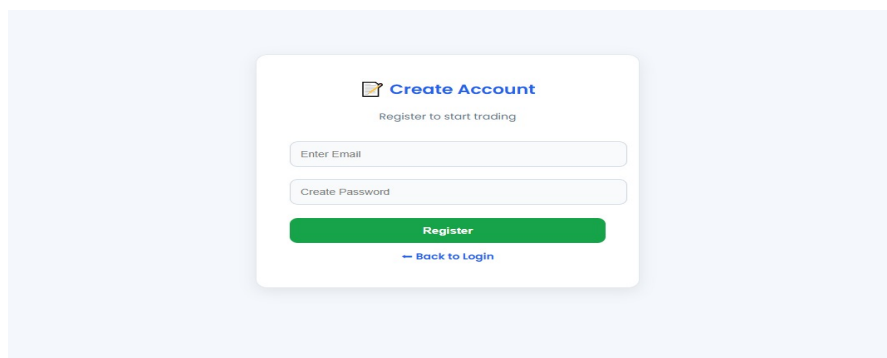


Figure 6.1.2: Register Page

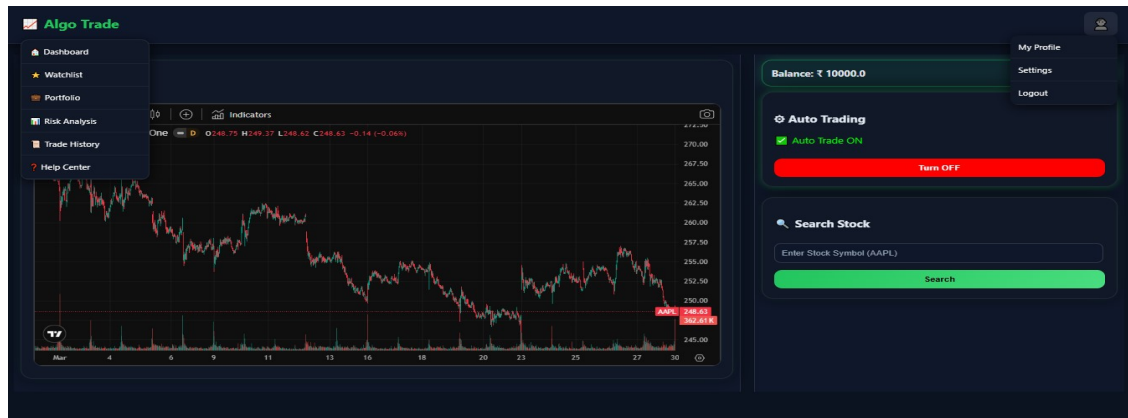


Figure 6.1.3: Dashboard Page

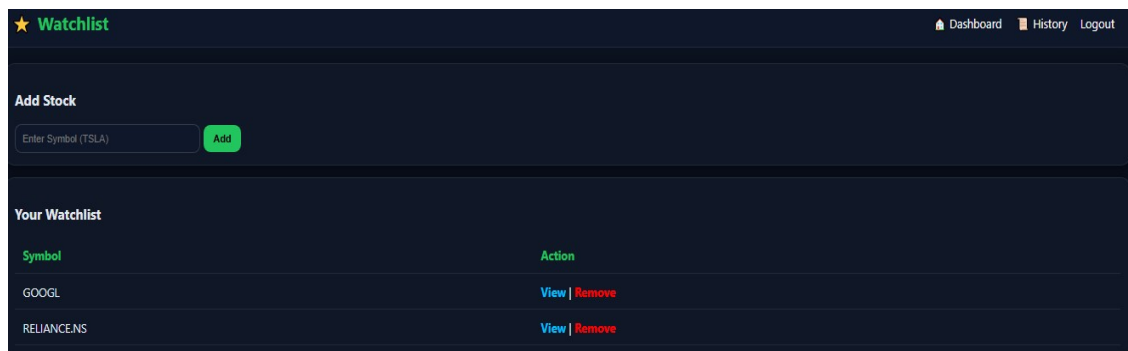


Figure 6.1.4: Watchlist Page

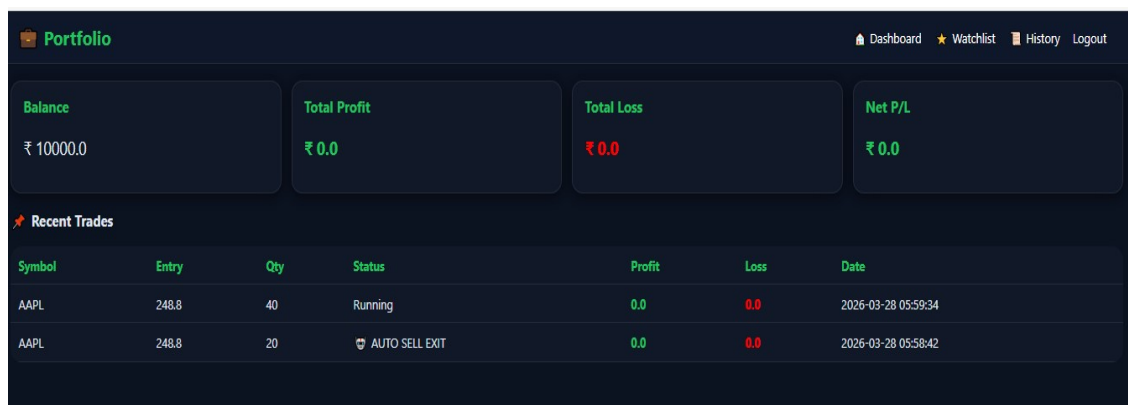


Figure 6.1.5: Portfolio Page

Trade History Dashboard Download CSV

Total Trades: 2

Filter: All

Total Profit
 ₹ 0.0

Total Loss
 ₹ 0.0

Net Result
 ₹ 0.0

Stock	Entry	Qty	SL	Target	Status	Profit	Loss	Date
AAPL	₹ 248.8	40	₹ 3600.0	₹ 234567.0	Running	₹ 0.0	₹ 0.0	2026-03-28 05:59:34
AAPL	₹ 248.8	20	₹ 220.0	₹ 260.0	AUTO SELL EXIT	₹ 0.0	₹ 0.0	2026-03-28 05:58:42

Figure 6.16: Trade History Page

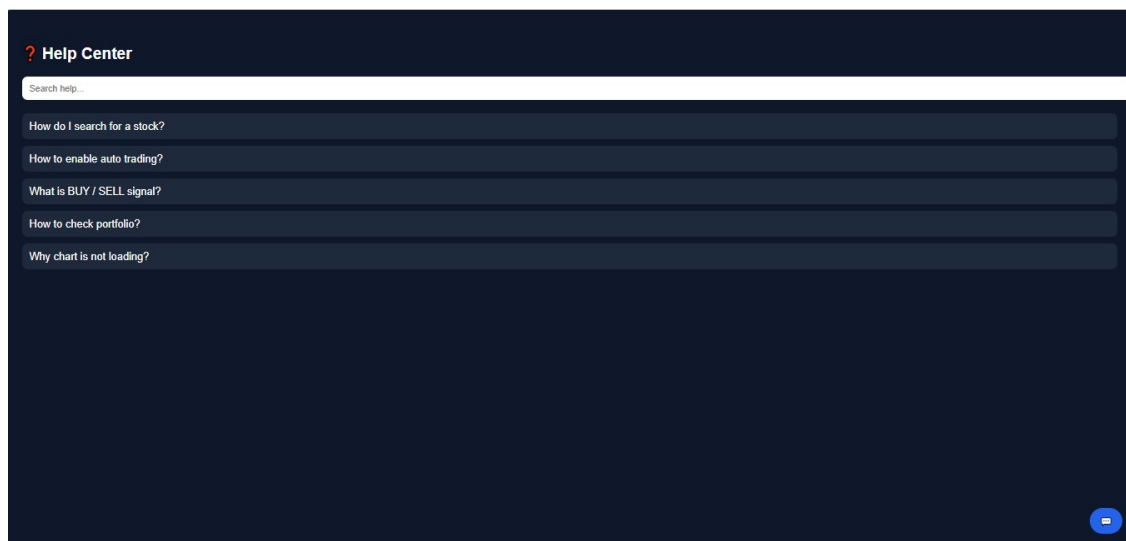


Figure 6.1.7: Help Center Page

6.2 API Integration Module

This module is used to fetch real-time stock market data from external sources. Yahoo Finance API (yfinance) is used for stock price, open, high, low, and volume details. TradingView widget is used for live chart display.

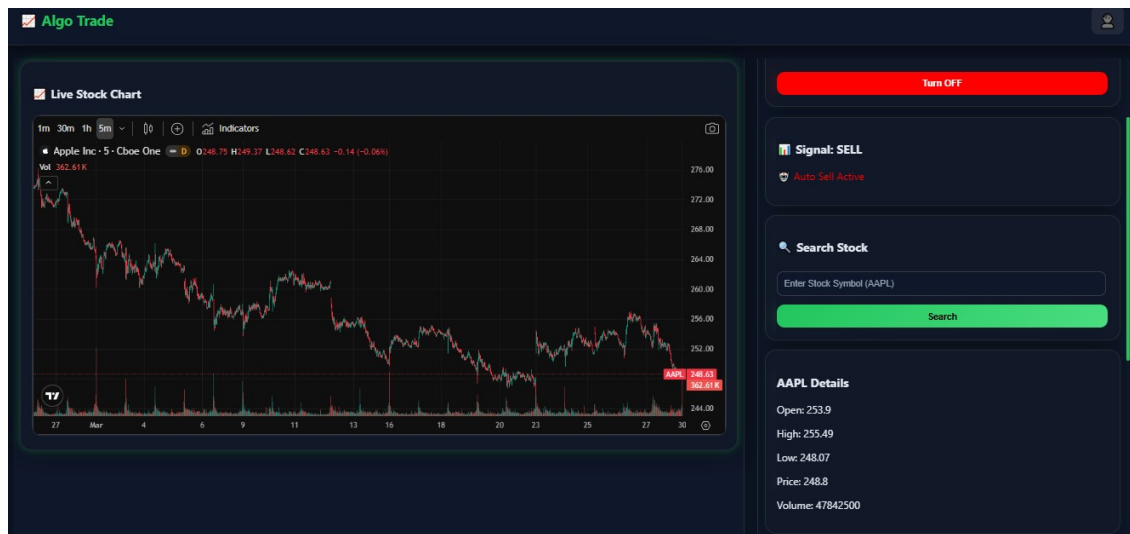


Figure 6.2: API Stock Data Fetch Screen

6.3 Data Processing Module

The Data Processing module cleans and formats the stock data received from the API. It calculates important values such as price change, percentage change, and prediction output. This processed data is used for monitoring and display.

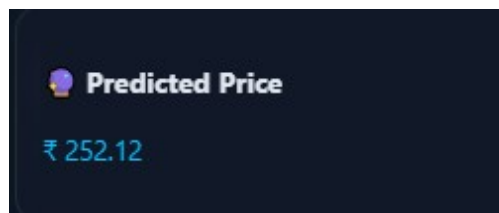


Figure 6.3: Processed Data and Prediction Output

6.4 Stock Monitoring Module

This module continuously monitors stock prices and compares current price with previous price. It calculates profit/loss and supports stop loss and target-based monitoring. It also provides auto sell exit option.

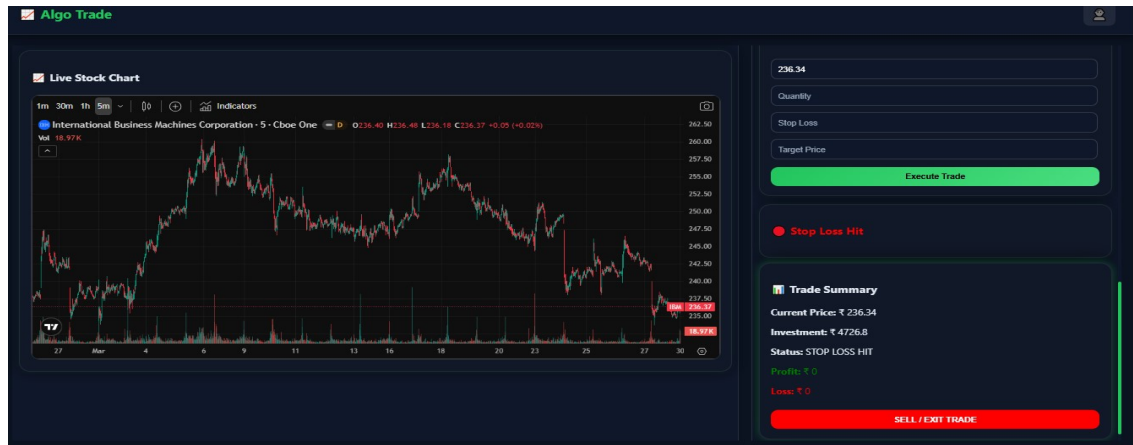


Figure 6.4: Execute Trade and Monitoring Screen

6.5 Display and Visualization Module

This module displays stock data in the form of charts and tables. It provides live stock charts, profit/loss details, and risk analysis summary. Graphs help users understand stock trends easily.

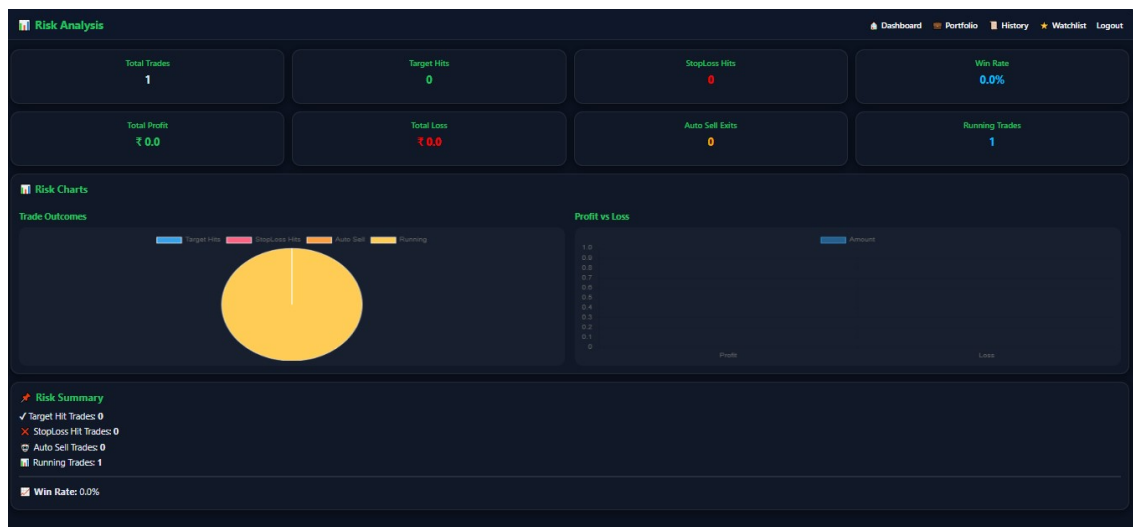


Figure 6.5: Risk Analysis Visualization Screen

6.6 Notification Module

The Notification module is used to alert the user when stock prices reach important levels such as target price, stop loss, or sudden price changes. It helps the user take quick action by showing popup messages or alert notifications on the dashboard.

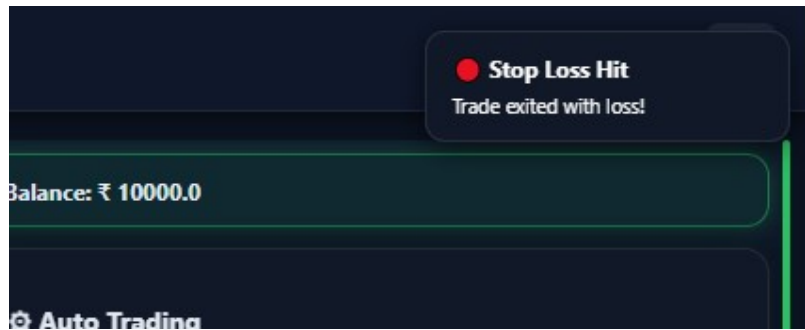


Figure 6.6: Notification Alert Screen

6.7 Profile and Settings Module

The Profile and Settings module allows users to view and update their account details such as username, email, and password. It also provides settings options to manage account preferences and application features.

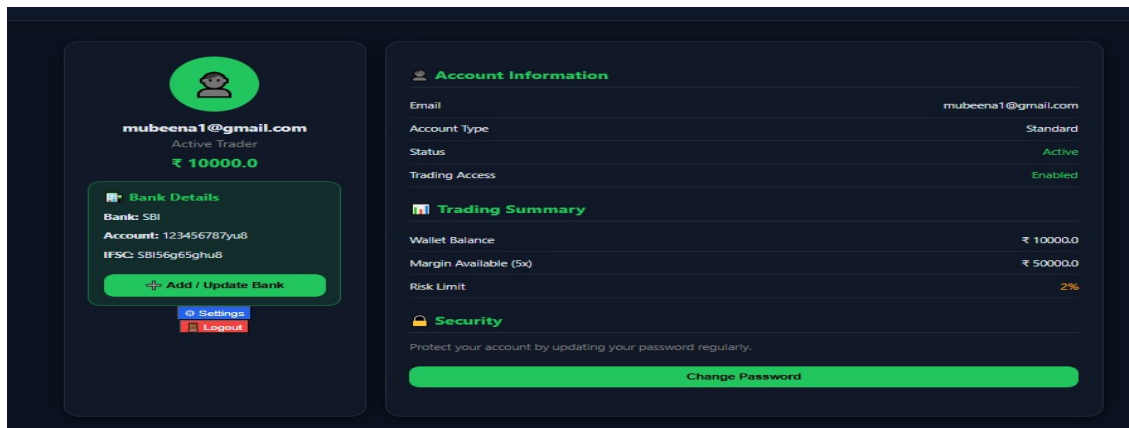


Figure 6.7.1: Profile Page Screen

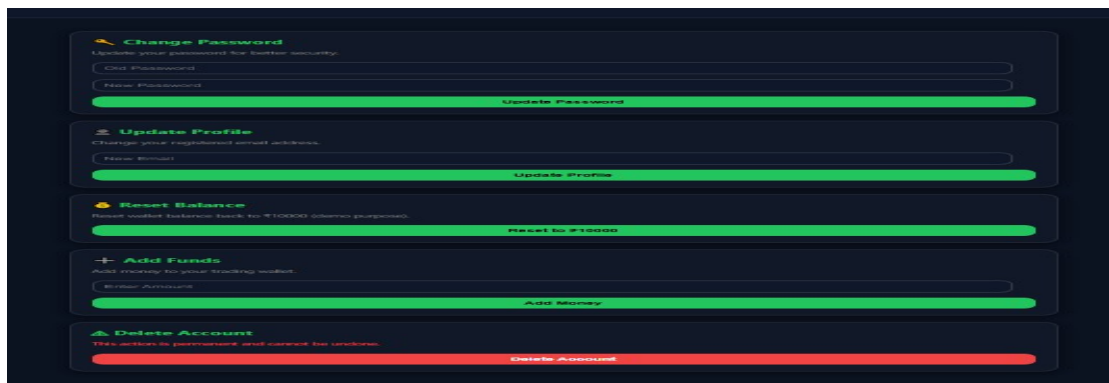


Figure 6.7.2: Settings Page Screen

6.8 SAMPLE SOURCE CODE

6.8.1 APP.PY

```

from flask import Flask, render_template, request, redirect, session
import yfinance as yf
import matplotlib.pyplot as plt
import os
import sqlite3
import numpy as np
from flask import Response
from sklearn.linear_model import LinearRegression
app = Flask(__name__)
app.secret_key = "Q8FU4PMTOKAW2RJ1"
def get_db():
    return sqlite3.connect("database.db")
def get_balance():
    user_id = session.get("user_id")
    return data[0] if data else 10000
def fix_symbol(symbol):
    symbol = symbol.upper()
stock = yf.Ticker(symbol + ".NS")
    data = stock.history(period="1d")
    if not data.empty:
        return symbol + ".NS"
stock = yf.Ticker(symbol)
    data = stock.history(period="1d")
    if not data.empty:
        return symbol
return None
def get_stock_data(symbol):
    try:
        stock = yf.Ticker(symbol)
        data = stock.history(period="1d")
    if data.empty and not symbol.endswith(".NS"):
        stock = yf.Ticker(symbol + ".NS")
        data = stock.history(period="1d")
        symbol = symbol + ".NS"
    if data.empty:
        return None

```

```

return {
    "symbol": symbol.upper(),
    "open": round(data["Open"].iloc[-1], 2),
    "high": round(data["High"].iloc[-1], 2),
    "low": round(data["Low"].iloc[-1], 2),
    "price": round(data["Close"].iloc[-1], 2),
    "volume": int(data["Volume"].iloc[-1])
}
except:
    return None
def predict_price(symbol):
    try:
        stock = yf.Ticker(symbol)
        data = stock.history(period="7d")
    if data.empty:
        return None
    data = data.dropna()
    if len(data) < 5:
        return None
    data["Days"] = np.arange(len(data))
    X = data[["Days"]]
    y = data["Close"]

    model = LinearRegression()
    model.fit(X, y)

    future = [[len(data)]]
    predicted = model.predict(future)[0]

    return round(predicted, 2)
except:
    return None
def get_signal(symbol):
    try:
        stock = yf.Ticker(symbol)
        data = stock.history(period="5d")
    if len(data) < 3:
        return "HOLD"
    prices = data["Close"]

```

```

if prices.iloc[-1] > prices.mean():
    return "BUY"
elif prices.iloc[-1] < prices.mean():
    return "SELL"
else:
    return "HOLD"
except:
    return "HOLD"
@app.route("/dashboard")
def dashboard():
    if "user_id" not in session:
        return redirect("/login")
    balance = get_balance()
    trade = session.get("trade")
    error_msg = session.pop("error_msg", None)
    stock = None
    investment = None
    profit = 0
    loss = 0
    current_price = None
    trade_status = None
    max_qty = None
    trade_status = "
```

```

    session["last_symbol"] = fixed_symbol
    return redirect("/dashboard")
    entry = trade["entry_price"]
    qty = trade["quantity"]
    pnl = (current_price - entry) * qty
    status = "RUNNING"
    if current_price <= trade["stop_loss"]:
        status = "STOP LOSS HIT"
    elif current_price >= trade["take_profit"]: status = "TARGET HIT"
    return {"status": status, "price": current_price, "profit": round(pnl, 2)}
@app.route("/profile")
def profile():
    if "user_id" not in session:
        return redirect("/login")
@app.route("/logout")
def logout():
    session.clear()
    return redirect("/login")
if __name__ == "__main__":
    app.run(debug=True)

```

6.8.2 DASHBOARD CODE

```

<!DOCTYPE html>
<html>
<head>
<title>STOCK TRACKER</title>
<link rel="stylesheet" href="/static/style.css">
<link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600&displ
ay=swap" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>>
</head>
<body>
<div
id="notifyBox"
style="display:none;position:fixed;top:20px;right:20px;background:#111827;co
lor:white;padding:15px 18px;border-radius:14px;box-shadow:0 4px 20px
rgba(0,0,0,0.6);border:1px solid rgba(255,255,255,0.1);z-index:9999;min-
width:220px;">
<b id="notifyTitle">Notification</b>

```

```

<p id="notifyMsg" style="margin:6px 0 0;font-size:14px;"></p>
</div>
<div class="navbar">
<div class="logoMenu">
<h2 onclick="toggleLogoMenu()" style="cursor:pointer;"><img alt="checkmark icon" data-bbox="665 170 685 190"/> Algo Trade</h2>
<div class="logoDropdown" id="logoDropdown">
<a href="/dashboard"><img alt="house icon" data-bbox="355 215 375 235"/> Dashboard</a>
<a href="/watchlist"><img alt="star icon" data-bbox="345 240 365 260"/> Watchlist</a>
<a href="/portfolio"><img alt="briefcase icon" data-bbox="340 265 365 285"/> Portfolio</a>
<a href="/risk_analysis"><img alt="bar chart icon" data-bbox="375 290 395 310"/> Risk Analysis</a>
<a href="/trade_history"><img alt="document icon" data-bbox="380 315 400 335"/> Trade History</a>
<a href="/help"><img alt="question mark icon" data-bbox="300 340 320 360"/> Help Center</a>
</div>
</div>
<div class="dashboard">
<div class="left">
<div class="card">
<h3><img alt="checkmark icon" data-bbox="200 460 220 480"/> Live Stock Chart</h3>
<div style="margin-top:20px;">
<iframe src="https://s.tradingview.com/widgetembed/?symbol={% if stock
%}{{stock.symbol.replace('.NS','')}}{% else %}AAPL{% endif
%}&interval=5&theme=dark" width="100%" height="500" frameborder="0"
style="border-radius:15px;border:1px solid rgba(255,255,255,0.1);"></iframe>
</div>
</div>
</div>
<div class="right">
<div class="balance"><b>Balance: ₹ {{balance}}</b></div>
<div class="card">
{% if signal %}
<div class="card">
<h3><img alt="bar chart icon" data-bbox="200 755 220 775"/> Signal: {{ signal }}</h3>
{% if signal == "BUY" %}
<p style="color:green;"><img alt="bull icon" data-bbox="370 795 390 815"/> Auto Buy Active</p>
{% elif signal == "SELL" %}
<p style="color:red;"><img alt="bear icon" data-bbox="350 840 370 860"/> Auto Sell Active</p>
{% else %}

```

```

<p> HOLD</p>
{% endif %}
</div>
{% endif %}
<div class="card">
<h3> Search Stock</h3>
<form action="/search" method="POST">
<input type="text" name="symbol" placeholder="Enter Stock Symbol (AAPL)"
required>
<br>
<button type="submit">Search</button>
</form>
</div>
{% if stock %}
<div class="card">
<h3>{{stock.symbol}} Details</h3>
<p>Open: {{stock.open}}</p>
<p>High: {{stock.high}}</p>
<p>Low: {{stock.low}}</p>
<p>Price: {{stock.price}}</p>
<p>Volume: {{stock.volume}}</p>
</div>
{% endif %}
{% if predicted_price is not none %}
<div class="card">
<h3> Predicted Price</h3>
<p style="color:deepskyblue;font-size:18px;">₹ {{ predicted_price }}</p>
</div>
{% endif %}
{% if stock %}
<div class="card">
<h3> Execute Trade</h3>
<form action="/trade" method="POST">
<input type="hidden" name="symbol" value="{{stock.symbol}}">
<input type="number" step="0.01" name="entry_price"
value="{{stock.price}}" readonly>
<input type="number" name="quantity" placeholder="Quantity" required>
<input type="number" step="0.01" name="stop_loss" placeholder="Stop Loss"
required>

```

```

<input type="number" step="0.01" name="take_profit" placeholder="Target
Price" required>
{% endif %}
{% if trade_status %}
<div class="card">
{% if 'Stop' in trade_status %}
<h3 style="color:red;">{{trade_status}}</h3>
{% elif 'Target' in trade_status %}
<h3 style="color:green;">{{trade_status}}</h3>
{% elif 'AUTO BUY' in trade_status %}
<h3 style="color:green;">{{trade_status}}</h3>
{% elif 'AUTO SELL' in trade_status %}
<h3 style="color:orange;">{{trade_status}}</h3>
{% else %}
<h3 style="color:blue;">{{trade_status}}</h3>
{% endif %}
</div>
{% endif %}
{% if investment %}
<div class="card">
<h3><img alt="Bar chart icon" data-bbox="198 501 225 521"/> Trade Summary</h3>
<p><b>Current Price:</b> ₹ <span id="livePrice">{{current_price}}</span></p>
<p><b>Investment:</b> ₹ {{investment}}</p>
<p><b>Status:</b> <span id="liveStatus">{{trade_status}}</span></p>
<p style="color:green;"><b>Profit:</b> ₹ <span id="liveProfit">{{profit}}</span></p>
<p style="color:red;"><b>Loss:</b> ₹ {{loss}}</p>
<form action="/sell" method="POST">
<button style="background:red;color:white;margin-top:10px;">SELL / EXIT
TRADE</button>
</form>
</div>
{% endif %}
</div>
</div>
<script>
document.getElementById("liveProfit").innerText=data.profit;
if(data.status!==lastStatus && data.status!=="RUNNING"){

```

```
lastStatus=data.status;
if(data.status=="TARGET HIT"){showNotification("☐ Target Hit","Trade
exited with profit!");}
else if(data.status=="STOP LOSS HIT"){showNotification("● Stop Loss
Hit","Trade exited with loss!");}
}
});
}
setInterval(updateLiveData,5000);
</script>
</body>
</html>
```

CHAPTER 7

PROJECT STRUCTURE & CODE IMPLEMENTATION

7. PROJECT STRUCTURE & CODE IMPLEMENTATION

This chapter explains the complete Flutter project structure and how the code is implemented in the Stock Tracker application. It includes the directory layout, state management approach, API integration with JSON parsing, and the overall flow of the application from login to dashboard and stock tracking.

7.1 Flutter Project Directory Structure

The Flutter project directory structure is designed to keep the application clean, organized, and easy to maintain. A proper folder structure helps in separating UI screens, models, API services, and business logic. This improves readability and makes the project scalable for future upgrades.

A typical Flutter project contains multiple folders such as **android**, **ios**, **assets**, and **lib**. The most important folder is the **lib** folder, where the entire Dart code is written. Inside the lib folder, the project is divided into different subfolders such as **screens**, **models**, **services**, **widgets**, and **utils**.

The **main.dart** file is the entry point of the Flutter application. It initializes the app, sets up themes, and loads the first screen such as Splash Screen or Login Screen.

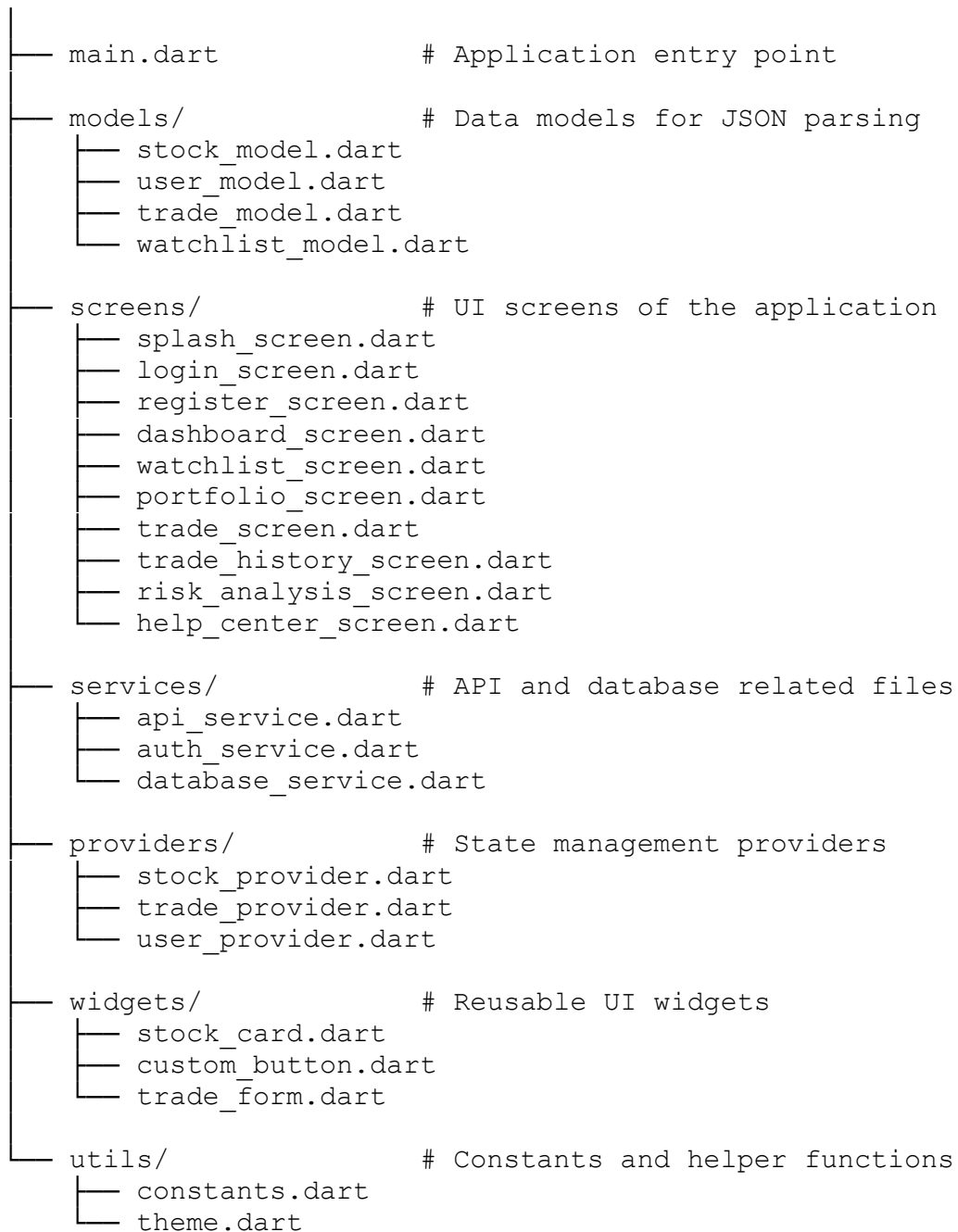
The **assets** folder contains images, icons, and fonts used in the application. These assets are configured in the **pubspec.yaml** file. The pubspec.yaml file is also used for managing external dependencies such as http package, provider package, and chart libraries.

The **android** and **ios** folders contain platform-specific configuration files required to run the application on Android and iOS devices. These include permissions, build settings, and application configurations.

In this Stock Tracker project, the directory structure is organized in a modular way where stock fetching, trading logic, watchlist management, and prediction features are separated into different modules. This approach makes the project more maintainable and easier to debug.

Directory Layout

```
stock_tracker/
├── android/                # Android-specific configuration
│   and permissions
├── ios/                    # iOS-specific configuration and
│   permissions
├── assets/                 # Images, icons, fonts
└── lib/                   # Main Flutter source code
```



7.2 State Management Implementation

State management is an important part of the Flutter application because it controls how data updates are reflected on the UI. In the Stock Tracker application, state management is used to update stock prices, manage watchlist, store portfolio details, and display trade history dynamically.

In this project, **Provider package** is used for state management. Provider helps in managing global data across the application without passing variables repeatedly between widgets.

Implementation Details

- **StockProvider** is used to fetch and store live stock data.
- **TradeProvider** is used to manage executed trades, profit/loss calculation, and trade history.
- **UserProvider** is used to manage user login session and profile details.

When stock data is fetched from the API, the provider updates the state using `notifyListeners()`. As a result, the UI updates automatically without manual refresh.

Advantages of Provider State Management

- Avoids unnecessary code repetition
- Improves performance
- Keeps UI and business logic separated
- Makes application scalable for future features like auto trading

Thus, state management plays a key role in keeping the Stock Tracker app responsive and interactive.

7.3 API Integration & JSON Parsing

The Stock Tracker application relies on external stock market data APIs to fetch live stock prices and historical stock information. API integration is implemented using the Flutter **http package**.

When the user enters a stock symbol (example: TSLA, AAPL, MSFT), the app sends an HTTP request to the stock API. The API returns the response in JSON format. This JSON data includes values like current price, open price, high price, low price, volume, and historical prices.

API Integration Process

1. User enters stock symbol in dashboard search bar
2. Flutter sends API request using `http.get()`
3. API returns JSON response
4. JSON is parsed into model objects
5. UI displays the stock chart and details

JSON Parsing

To avoid handling raw JSON maps directly, the application uses model classes such as `StockModel`. These models contain factory constructors like:

- `StockModel.fromJson(Map<String, dynamic> json)`

This method converts JSON values into structured Dart objects. This makes the code clean and reduces errors.

Error Handling

To avoid crashes due to poor network connection, API calls are handled using `try-catch` blocks. If API fails, the system displays a message like:

- "Network Error"
- "Invalid Stock Symbol"
- "API Not Responding"

This module ensures that real-time stock data is fetched safely and displayed properly in the application.

7.4 Main Application Flow

The main flow of the Stock Tracker application explains how the system works from launching the app to tracking stock performance.

Step-by-Step Application Flow

- 1. App Launch**
 - The application starts from `main.dart`.
 - Flutter initializes all required plugins and packages.
- 2. Splash Screen**
 - The app first displays the splash screen.
 - It checks whether the user is already logged in or not.
- 3. Login/Register**
 - If user is not logged in, the app navigates to login screen.
 - New users can create an account using register screen.
- 4. Dashboard Loading**
 - After successful login, the dashboard screen is displayed.
 - User can search stock symbol and view stock chart.
- 5. Stock Data Fetching**
 - The API service sends request to stock API.
 - The fetched JSON response is processed and stored in `StockProvider`.

6. Watchlist and Portfolio

- User can add stocks to watchlist.
- User can view portfolio holdings and investment summary.

7. Trade Execution

- User can execute trade by selecting quantity, stop loss, and target.
- The trade is stored in database and shown in trade history.

8. Risk Analysis and Notifications

- Risk analysis page displays trade performance summary.
- Notifications alert user when target/stop loss is reached.

9. Logout

- User can logout using profile option.
- Session is cleared and redirected back to login page.

This flow ensures that the Stock Tracker application provides smooth navigation, live stock tracking, and trading analysis features in a user-friendly manner.

CHAPTER 8

TESTING

8.TESTING

Software testing is an important phase in the Software Development Life Cycle (SDLC). It ensures that the Stock Tracker application works correctly, provides accurate results, and handles errors properly. Since the Stock Tracker system deals with real-time stock market data, trading actions, and profit/loss calculations, testing is required to validate accuracy, reliability, and smooth user experience.

The testing process helps identify bugs, improve system performance, and ensure the application functions correctly under different conditions such as invalid stock symbols, network errors, and wrong user inputs.

8.1 Testing Objectives

The main objectives of testing the Stock Tracker Basic application are:

1. **Validate Functional Requirements**
Ensure that all major modules such as Login/Register, Dashboard, Stock Search, Watchlist, Portfolio, Trade Execution, Trade History, and Risk Analysis work correctly.
2. **Verify Data Accuracy**
Ensure that the stock price values fetched from API (Open, High, Low, Volume, Current Price) are displayed correctly without errors.
3. **Check Trade Logic Correctness**
Verify that profit/loss calculation, target hit, stop loss hit, and auto sell exit features are working properly.
4. **Ensure UI Consistency**
Confirm that all pages display properly with correct navigation buttons, charts, and responsive layout.
5. **Test Error Handling**
Ensure the system behaves properly during errors such as invalid stock symbol, API failure, or no internet connection.

8.2 Types of Testing Performed

To ensure full coverage, different types of testing were performed on the Stock Tracker system.

8.2.1 Unit Testing

Unit testing is performed to test individual functions and modules separately. It checks whether a function produces correct output for given input.

Target Areas:

- Profit and loss calculation function
- Prediction model output (Linear Regression)
- Stop loss and target checking logic

- Database insert and fetch functions

Example:

A unit test was performed by passing stock buy price = 100 and current price = 120. The expected profit should be 20 and the function returned correct value.

8.2.2 Integration Testing

Integration testing checks whether different modules work together correctly. It ensures smooth communication between frontend pages, Flask backend, database, and API.

Target Areas:

- Login page connected with database validation
- Stock search connected with API fetch
- Trade execution connected with trade history update
- Watchlist connected with database storage

Example:

After executing a trade, the system was tested to ensure that the trade is automatically stored in trade history and portfolio updates correctly.

8.2.3 System Testing

System testing checks the complete application as a whole. It validates the end-to-end flow from user login to stock monitoring and risk analysis.

Target Areas:

- Full application workflow
- User navigation across all pages
- Live stock tracking
- Auto trading ON/OFF flow
- Risk analysis report generation

Example:

A test was performed where the user logs in, searches a stock, executes a trade, and checks trade history and risk analysis output.

8.2.4 UI Testing

UI testing checks the design and layout of the application. It ensures all pages are displayed correctly and navigation works smoothly.

Target Areas:

- Login/Register page alignment
- Dashboard layout and stock chart display
- Sidebar navigation menu

- Buttons like Add Watchlist, Execute Trade, Auto Trading ON/OFF

Example:

The dashboard was tested on different screen sizes to ensure the stock chart and data table are displayed properly.

8.2.5 API Testing

API testing is done to verify whether stock market data is fetched correctly. Since the system depends on external API, this testing is very important.

Target Areas:

- Correct stock symbol fetch
- Correct values displayed (Open, High, Low, Volume)
- Handling invalid symbols
- Handling API timeout errors

Example:

When the user searches “AAPL”, the system correctly displays the Apple stock details. When the user enters an invalid symbol, the system shows an error message instead of crashing.

8.3 Sample Test Cases

Below table shows some test cases executed during the testing phase:

Test Case ID	Feature/Module	Test Description	Expected Output	Actual Output	Status
TC_1	Login Module	Enter valid username & password	User should login successfully	Login successful	PASS
TC_2	Login Module	Enter wrong password	Show error message	Error displayed	PASS
TC_3	Register Module	Register new user	Account should be created	Account created	PASS
TC_4	Stock Search	Enter valid stock symbol (AAPL)	Show stock price & chart	Displayed correctly	PASS
TC_5	Stock Search	Enter invalid symbol	Show invalid symbol error	Error displayed	PASS
TC_6	Watchlist	Add stock to watchlist	Stock should be saved	Added successfully	PASS
TC_7	Portfolio	Execute buy trade	Portfolio should update	Updated correctly	PASS
TC_8	Trade Execution	Enter target and	Trade should	Working	PASS

		stop loss	execute properly	correctly	
TC_9	Auto Trading	Turn ON auto trade	Auto trade should activate	Activated successfully	PASS
TC_10	Risk Analysis	Check win/loss summary	Should show correct values	Displayed correctly	PASS
TC_11	API Error Handling	Disconnect internet and fetch stock	Show network error	Network error shown	PASS
TC_12	Trade History	After trade execution	Trade should be stored	Stored correctly	PASS

8.4 Testing Result Summary

All the modules in the Stock Tracker Basic system were tested successfully. Login and registration modules worked correctly with database authentication. API integration successfully fetched stock data and displayed real-time charts. Trade execution and monitoring features correctly calculated profit/loss and updated portfolio and trade history. Risk analysis module generated accurate performance summary. The system also handled invalid stock symbols and network errors smoothly without crashing.

Thus, testing confirms that the Stock Tracker Basic application is stable, reliable, and ready for deployment.

CHAPTER 9

SYSTEM RESULTS AND EVALUATION

9. SYSTEM RESULTS AND EVALUATION

The Stock Tracker Basic application was successfully developed, executed, and tested in a web-based environment using Python Flask. The final system provides a complete platform for monitoring real-time stock market data, tracking user trades, and displaying performance analysis. The overall results and system performance are evaluated below.

9.1 Fulfillment of Objectives

The main objective of this project was to develop a simple and user-friendly stock tracking system that can fetch live stock data, support trade monitoring, and provide risk analysis features. The system successfully achieved the predefined objectives:

- **Accurate Stock Data Fetching:**
The application successfully integrates with the Yahoo Finance API (yfinance) to fetch real-time stock details such as Open, High, Low, Current Price, and Volume.
- **User-Friendly Dashboard:**
The dashboard module provides an interactive interface with stock charts, search option, and navigation menus such as Watchlist, Portfolio, Risk Analysis, and Trade History.
- **Trade Execution and Monitoring:**
The system supports trade execution with stop loss and target features. It correctly monitors the trade status and calculates profit/loss values.
- **Auto Trading Support:**
The Auto Trading ON/OFF option works successfully and allows the user to enable or disable automatic trading mode.
- **Risk Analysis and Trade Summary:**
The risk analysis module successfully generates performance results such as total trades, total profit, total loss, win rate, and stop loss/target hit count.

9.2 Performance and Efficiency Analysis

Performance evaluation is important because the system depends on real-time stock data and continuous monitoring.

- **API Response and Data Loading:**
The stock data is fetched efficiently from the API and displayed on the dashboard within a few seconds depending on network speed.
- **Graph and Visualization Performance:**
The system successfully generates stock price charts and risk analysis graphs without delay. Visualization output is clear and easy to understand.
- **Database Storage and Retrieval:**
Trade history, watchlist, and portfolio data are stored in the database properly. Retrieval of stored records is fast and accurate.
- **System Stability:**
The application handles invalid stock symbols and network errors gracefully by displaying error messages instead of crashing.

9.3 Overall Application Outcome

The final Stock Tracker Basic system is a fully functional application that helps users track stock prices, monitor trades, and analyze profit/loss performance. Unlike traditional stock platforms that may be complex for beginners, this system is designed with a simple interface and easy navigation.

The modular design ensures that the application is scalable, meaning additional features such as advanced prediction models, real-time notifications, and auto buy/sell execution can be implemented in the future. The system successfully meets project requirements and provides a reliable platform for stock monitoring and analysis.

CHAPTER 10

FUTURE ENHANCEMENTS

10. FUTURE ENHANCEMENTS

The Stock Tracker project application can be further improved by adding advanced features and technologies to enhance its performance, accuracy, and usability. The following enhancements can be considered for future development:

1. **Advanced Machine Learning Models:** More sophisticated algorithms such as Decision Trees, Random Forest, or Neural Networks can be used to improve prediction accuracy.
2. **Real-Time Notifications:** The system can be enhanced to send alerts and notifications for stock price changes, buy/sell signals, and market updates.
3. **Mobile Application Development:** A mobile version of the application can be developed to provide easy access for users on smartphones.
4. **User Authentication System:** Login and registration features can be added to provide personalized user experience and data security.
5. **Portfolio Management:** Users can be allowed to track their investments, profits, and losses within the system.
6. **Integration with Trading Platforms:** The system can be connected to real trading platforms to enable actual buying and selling of stocks.
7. **Improved Visualization:** Advanced charts and dashboards can be added for better data representation and analysis.
8. **Cloud Integration:** Cloud-based storage and deployment can be implemented for better scalability and accessibility.

CHAPTER 11

CONCLUSION

11. Conclusion

The Stock Tracker project application has been successfully developed to provide an efficient and user-friendly platform for tracking and analyzing stock market data. The system is capable of fetching real-time stock information, displaying key details, and performing basic analysis using techniques such as moving averages. It also incorporates a prediction feature using Linear Regression, which helps users estimate future stock prices.

The application simplifies stock market analysis for beginners and students by presenting data in a clear and understandable format. Features such as auto buy and auto sell suggestions further assist users in decision-making. Although the system provides useful insights, it is limited by the simplicity of the prediction model and dependence on external APIs.

Overall, the project meets its objectives and demonstrates the practical use of programming, data analysis, and machine learning in real-world applications. With future enhancements, the system can be made more accurate, scalable, and suitable for advanced users.

CHAPTER 12

REFERENCES

12. REFERENCES

1. Yahoo Finance API (yfinance) Documentation

<https://pypi.org/project/yfinance/>

2. Python Official Documentation:

<https://docs.python.org/3/>

3. pandas Library Documentation:

<https://pandas.pydata.org/docs/>

4. numpy Library Documentation:

<https://numpy.org/doc/>

5. matplotlib Library Documentation:

<https://matplotlib.org/stable/contents.html>

- 6.scikit-learn Library Documentation:

<https://scikit-learn.org/stable/>

- 7.Stock Market Analysis Tutorials (General Reference):

<https://www.investopedia.com/stock-analysis-4689748>